Comprehensive Guide to the Extended Kalman Filter (EKF)

Imagine you are on a commercial airplane flying through a thick storm. The pilots cannot see outside, and GPS signals can occasionally become weak or blocked. How does the autopilot system still track the plane's exact position, speed, and orientation? It uses math to blend its last known location with noisy data from the plane's internal sensors. This magic blending trick is called a **Kalman Filter**.

In the real world, airplanes don't just move in perfectly straight lines—they turn, climb, descend, and move in complex curves. To track these curved movements, we use a special upgraded version called the **Extended Kalman Filter (EKF)**.

To understand how this works, we will build the math from the ground up, starting with simple high school algebra, moving through calculus, and finishing with advanced linear algebra. We will weave all the foundational pieces into one smooth story.

Part 1: The Foundations of Measurement

Before we can build a filter, we must understand how to measure things and how to measure our own uncertainty.

1. Variables in Multiple Dimensions (Matrices and Vectors)

If we only track a car's forward position, we use a single number (a scalar) like $x = 5$. But cars have speed, direction, and 2D coordinates. * **Algebra Format:** A single equation: $y = 3 \cdot x$ * **Calculus Format:** A velocity vector is the derivative of a position vector: $v(t) = [x'(t), y'(t)]$. * **Advanced Math (Linear Algebra) Format:** We group multiple variables into a list called a **vector**. We use a grid of numbers called a **matrix** to manipulate them all at once. Instead of $y = 3x$, we write $Y = A \cdot X$. We multiply the rows of matrix A by the column of vector X to update all our variables simultaneously.

2. Measuring Uncertainty (Variance and Covariance)

Sensors aren't perfect. We need a way to measure how much we *don't* know. * **Algebra Format: Variance** measures how spread out a single variable is from its average (μ). $Var(x) =$ Average of $(x - \mu)^2$. **Covariance** measures how two variables move together. If position and speed increase together, their covariance is positive. $Cov(x, y) =$ Average of $((x - \mu_x) \cdot (y - \mu_y))$. * **Calculus Format:** For continuous probability curves, we use integrals to find variance: $Var(x) = \int (x - \mu)^2 f(x) dx$ * **Advanced Math (Linear Algebra) Format:** We track the uncertainty of an entire vector using a **Covariance Matrix** (Σ or

P). It holds the variance of each variable on the diagonal and the covariances on the off-diagonals. $\Sigma = \begin{bmatrix} Var(x) & Cov(x,y) \\ Cov(y,x) & Var(y) \end{bmatrix}$

3. The Shape of Probability (Normal Distribution)

When sensor errors happen, they usually cluster around the true value in a bell-shaped curve. * **Algebra Format:** A 1D bell curve defined by a center (mean μ) and a width (standard deviation σ). * **Calculus Format:** The exact probability density function: $f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}(\frac{x-\mu}{\sigma})^2}$ * **Advanced Math (Linear Algebra) Format:** A 3D probability cloud (Multivariate Normal Distribution). We replace division by variance σ^2 with multiplication by the inverse of the covariance matrix Σ^{-1}. $f(X) \propto \exp\left(-\frac{1}{2}(X-\mu)^T \Sigma^{-1}(X-\mu)\right)$

Part 2: The Standard Linear Kalman Filter

Now we have the tools. Let's derive the standard Kalman Filter, which assumes everything moves in perfect straight lines. It happens in two steps: Predict and Correct.

Step 1: The Prediction

We guess where the car is based on its last state.

- **Algebra Format (1D):** New Position = (Multiplier · Old Position) + Noise $x_{new} = a \cdot x_{old} + w$ Uncertainty expands when we predict: $p_{new} = a^2 \cdot p_{old} + q$ (where q is process noise).
- **Calculus Format:** Using derivatives, if position $x(t) = v \cdot t$, the change over time is $\frac{dx}{dt} = v$. We use this constant rate to step forward in time.
- **Advanced Math (Linear Algebra) Format:** State Prediction: $X_{new} = A \cdot X_{old} + W$ Uncertainty Prediction: You can't just "square" a matrix A like we squared a in algebra. You multiply it by its transpose! $P_{new} = A \cdot P_{old} \cdot A^T + Q$

Step 2: The Correction

We read the GPS sensor. It gives us a measurement Z. The sensor reading relates to our true state by a multiplier H. How much do we trust the sensor versus our prediction? We calculate a ratio called the **Kalman Gain (K)**.

- **Algebra Format (1D):** $K = \frac{\text{Prediction Uncertainty}}{\text{Prediction Uncertainty+Sensor Noise (R)}}$ $K = \frac{p \cdot h}{h \cdot p \cdot h + R}$ Final State = Prediction + $K \cdot$(Actual Sensor − Expected Sensor) $x_{final} = x_{new} + K \cdot (z - h \cdot x_{new})$

- **Advanced Math (Linear Algebra) Format:** Kalman Gain (matrix division is multiplying by the inverse): $K = P_{new} \cdot H^T \cdot (H \cdot P_{new} \cdot H^T + R)^{-1}$

  Final State Update: $X_{final} = X_{new} + K \cdot (Z - H \cdot X_{new})$

  Final Uncertainty Update (our uncertainty shrinks because we added new info!): $P_{final} = (I - K \cdot H) \cdot P_{new}$ (where I is the identity matrix, acting like the number 1).

Part 3: Handling Curves (The EKF Upgrade)

The standard Kalman filter uses fixed matrices A and H. This strictly represents straight lines. But if our car turns, the math function $f(x)$ becomes curvy (non-linear). To use our Kalman equations, we must straighten out the curves!

1. Slopes and Partial Derivatives

To straighten a curve, we need to know its slope. * **Algebra Format:** Slope is rise over run. $m = \frac{y_2 - y_1}{x_2 - x_1}$ * **Calculus Format:** The derivative gives the exact slope at a point: $f'(x)$. A **partial derivative** $\frac{\partial f}{\partial x}$ gives the slope in one specific direction.

2. Straightening the Curve (Taylor Series)

- **Algebra Format:** We estimate a nearby point on a curve by drawing a straight line: $y_{est} = y_{start} + m \cdot (x - x_{start})$.
- **Calculus Format:** The Taylor Series proves this. $f(x) = f(a) + f'(a)(x - a) + \frac{f''(a)}{2}(x - a)^2 + \dots$ If we zoom in extremely close, $(x - a)$ is tiny. Squaring a tiny number makes it effectively zero. We can delete the rest of the equation! We are left with a perfect linear approximation: $f(x) \approx f(a) + f'(a)(x - a)$

3. The Multi-Variable Slope (The Jacobian Matrix)

Because our car has multiple variables, our derivative $f'(a)$ isn't a single number. We take all the partial derivatives of all our variables and pack them into a grid called the **Jacobian Matrix (J)**. * **Linear Algebra Format:** The linear approximation of a curvy vector function becomes: $F(X) \approx F(A) + J \cdot (X - A)$

Part 4: Deriving the Extended Kalman Filter

We now take our standard Kalman filter equations and apply our new Taylor Series Jacobian trick to them line by line!

The Prediction Phase (EKF)

- **Algebra Format:** Instead of a straight line $y = ax$, our state evolves by a curvy function $f(x)$. State: $x_{new} = f(x_{old})$ Uncertainty: We use the slope of the curve $f'(x)$. $p_{new} = (f'(x))^2 \cdot p_{old} + q$
- **Calculus Format:** By the 1st-order Taylor Series, we linearized $f(x)$ using its derivative $f'(x)$ evaluated at our last best guess. This derivative acts as our new, temporary straight-line multiplier.
- **Advanced Math (Linear Algebra) Format:** State: We just pass our vector through the non-linear math function: $X_{new} = f(X_{old})$ Uncertainty: We replace the fixed matrix A with the **Jacobian Matrix (F_k)** calculated at the current moment. $P_{new} = F_k \cdot P_{old} \cdot F_k^T + Q$

The Correction Phase (EKF)

- **Algebra Format:** Our sensor reading is also a curvy function $h(x)$. We use its slope $h'(x)$ to calculate the Kalman gain. $K = \frac{p \cdot h'(x)}{h'(x) \cdot p \cdot h'(x) + R}$
- **Calculus Format:** Again, using the Taylor Series, we approximate the curvy sensor function $h(x)$ using its first derivative $h'(x)$. We evaluate this derivative at our newly predicted state.
- **Advanced Math (Linear Algebra) Format:** We calculate the Jacobian Matrix of the sensor function, calling it H_k.

  Step-by-Step EKF Update:

  1. Calculate Kalman Gain using the Jacobian H_k: $K = P_{new} \cdot H_k^T \cdot (H_k \cdot P_{new} \cdot H_k^T + R)^{-1}$
  2. Update State (Note we use the true curvy function $h(X)$ to find the expected sensor reading, not the Jacobian!): $X_{final} = X_{new} + K \cdot (Z - h(X_{new}))$
  3. Update Uncertainty: $P_{final} = (I - K \cdot H_k) \cdot P_{new}$

Summary of the Journey

We started with basic slopes and single variables. We realized that to track complex things like cars, we needed Matrices and Covariance. We derived the standard Kalman Filter for perfectly straight movements. Then, by using Calculus (Derivatives and Taylor Series) to zoom in and straighten out curves, we upgraded our math to Linear Algebra (Jacobian Matrices). This birthed the Extended Kalman Filter, allowing us to perfectly blend noisy predictions and noisy sensors in a completely curved, non-linear world!